

Compilers

One Line of PyTorch, Many GPU Kernels

- `y = torch.relu(x @ w + b)`
- At the Python level, this looks like one simple expression.
- But in eager mode, PyTorch may execute it as multiple operations:

matmul kernel



add kernel =>



relu kernel =>

- Each step may read from and write back to GPU global memory.

What Actually Happens in Eager Mode?

- Eager Mode: Execute Immediately, Optimize Locally

`y = torch.relu(x @ w + b)`

- Eager execution may look like:

1. matmul

read x, read w

compute `x @ w`

write intermediate result to HBM

2. add

read intermediate result from HBM

read b

compute `x @ w + b`

write intermediate result to HBM

3. relu

read intermediate result from HBM

compute relu

write final output to HBM

Too much HBM reading and writing!!!!

Why Is This a Problem?

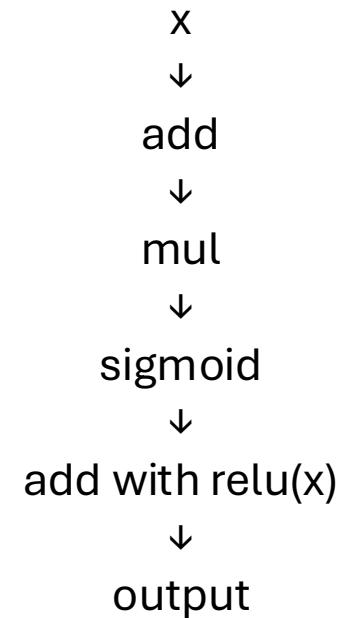
- The GPU Is Fast, But Memory Traffic Is Expensive
- Eager mode is flexible, but it can introduce overhead:
 - Kernel launch overhead
 - Python dispatch overhead
 - Intermediate tensor materialization
 - Repeated HBM read/write
 - Limited cross-operator optimization
- For small operations, launch overhead can dominate.
- For large tensors, HBM traffic can dominate.

Can We See the Whole Computation Before Running It?

- Instead of executing immediately:

add kernel
mul kernel
sigmoid kernel
relu kernel
add kernel

- we want to first capture the computation:
one fused kernel



This is the bridge to graph capture.

Graph Capture: Turning Python Execution into a Computation Graph

- Example Python code:

```
def f(x):
```

```
    y = x + 1
```

```
    y = y * 2
```

```
    y = torch.relu(y)
```

```
    return y
```

Captured graph:

$x \rightarrow \text{add} \rightarrow \text{multiply} \rightarrow \text{relu} \rightarrow \text{output}$

Once we have a graph, the compiler can apply:

Operator fusion

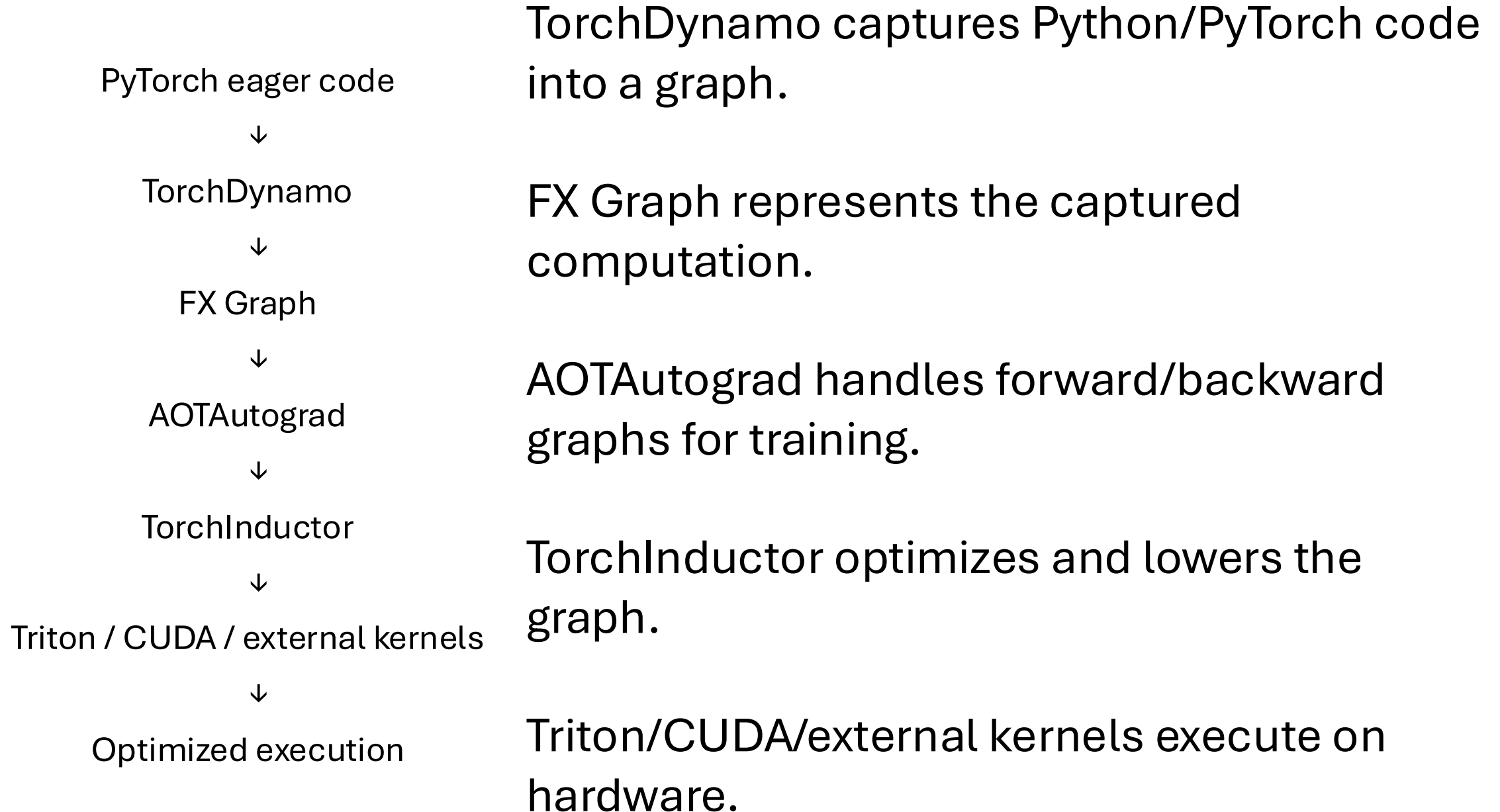
Memory planning

Decomposition

Lowering

Code generation

PyTorch Compiler Pipeline



TorchDynamo Is the Frontend

- TorchDynamo is the frontend of torch.compile.
- It intercepts Python execution and extracts PyTorch tensor operations into FX graphs.
- It works at the Python frame / bytecode level.
- This allows normal PyTorch code to stay mostly unchanged:
`compiled_model = torch.compile(model)`

TorchDynamo Is Not Just Ordinary Tracing

- Ordinary tracing usually runs the function once and records what happened.

```
def f(x, flag):  
    if flag:  
        return torch.relu(x)  
    else:  
        return torch.sigmoid(x)
```

- A normal trace may only record the branch taken by the example input.
- TorchDynamo works at the Python frame and bytecode level, so it can inspect Python execution more carefully, insert guards, and handle graph breaks.

The Compiler Wants a Continuous Graph

- For a simple tensor-only function:

```
def f(x):
```

```
    a = x + 1
```

```
    b = a * 2
```

```
    c = torch.relu(b)
```

```
    return c
```

- TorchDynamo can capture the whole function as one graph:
x → add → multiply → relu → output
- This is good for the compiler because the whole computation is visible.
- Once the full region is captured, TorchInductor can fuse operations, reduce memory traffic, and generate optimized code.

Graph Break: When Dynamo Has to Stop Capturing

- A graph break happens when TorchDynamo cannot safely continue capturing Python code into the same FX graph.

- Instead of one continuous graph:

$x \rightarrow \text{add} \rightarrow \text{multiply} \rightarrow \text{relu} \rightarrow \text{output}$

- Dynamo may split the execution into:

Graph 1: $x \rightarrow \text{add}$

Python eager execution

Graph 2: $\text{multiply} \rightarrow \text{relu} \rightarrow \text{output}$

- The program can still run correctly, but the compiler no longer sees the whole computation at once.

Example 1 — .item() Breaks the Tensor World

- Consider this function:

```
def f(x):
```

```
    if x.sum().item() > 0:
```

```
        return x * 2
```

```
    else:
```

```
        return x - 2
```

`x.sum().item()`

- `x.sum()` is a tensor operation.
- But `.item()` converts the tensor value into a Python scalar.
- Now the Python if statement depends on runtime tensor data.
- This is difficult to capture as a static graph.

Why .item() Is Expensive for Compilation

- When the code contains:

```
if x.sum().item() > 0:
```

- three things happen:
 - A GPU tensor value must be converted into a Python scalar.
 - Python control flow depends on that scalar.
 - The graph structure may depend on runtime data.

This may also force synchronization:

GPU computes `x.sum()`

↓

CPU/Python waits for the value

↓

Python decides which branch to run

This breaks the compiler's ability to optimize the whole computation as one graph.

How to Avoid This Graph Break

```
def f(x):  
    if x.sum().item() > 0:  
        return x * 2  
    else:  
        return x - 2
```

```
def f(x):  
    cond = x.sum() > 0  
    return torch.where(cond, x * 2, x - 2)
```

The computation can be represented as a graph:

x → sum → compare
x → multiply by 2
x → subtract 2
↓
torch.where
↓
output

Other Common Causes of Graph Breaks

- Graph breaks are often caused by Python behavior that cannot be safely represented inside an FX graph.

```
print(x.shape)
```

- Python I/O or logging inside the compiled region.

```
cache.append(x)
```

- Python side effects and mutation.

```
n = int(x[0].item())
```

```
for _ in range(n):
```

```
    x = x + 1
```

- Loop count depends on runtime tensor data.

```
some_external_library(x)
```

Performance Impact of Graph Breaks

- Graph breaks can hurt performance because they reduce the compiler's optimization scope.
- Without graph break:

add → multiply → sigmoid → relu → add

- Inductor may fuse the whole chain into one kernel.

- With graph break:

Graph 1: add → multiply

Python eager region

Graph 2: sigmoid → relu → add

Consequences:

Less operator fusion

More intermediate tensor
materialization

More kernel launches

More Python dispatch overhead

Possible CPU-GPU synchronization

More fragmented execution

Guards: Can We Reuse the Compiled Graph?

- When Dynamo compiles a graph, it makes assumptions:

Input shape

Input dtype

Input device

Tensor layout / stride

Module attributes

Python object state

`x.shape == [32, 768]`

`x.dtype == float16`

`x.device == cuda`

`x` is contiguous

Guards check whether these assumptions still hold on the next call.

Cache: Avoid Compiling the Same Graph Again

- First call:

capture graph
compile graph
generate code
store compiled version in cache
Execute

- Later call:

check guards
guards pass
reuse cached compiled version
execute

- If guards fail:

recompile
create a new cached version

TorchInductor: Optimize the Captured Graph

- TorchDynamo answers:

Can we capture this Python/PyTorch program as a graph?

- TorchInductor answers:

How do we make this graph run faster?

- TorchInductor performs:

Fusion
Decomposition
Memory planning
Lowering
Scheduling
Code generation

- On GPU, it often generates Triton kernels or calls external libraries such as cuBLAS.